Recap

Design Principles

From Code to High-Quality Software

Problem

- Knowing Language **Basics**, **OOP**, **DSA** alone doesn't guarantee **High Quality Software**

Bad Design		Consequences
Rigidity Small changes require many dependent changes	→	**Hard to Maintain**
Fragility Modifications cause unexpected bugs elsewhere	→	**Low Resilience**
Immobility Code can't be reused effectively	→	**Inflexible Systems**

src

Solution: Design Principles

- **What?**
  - High-level, language-independent guidelines and best practices for building high quality software
- **Why?**
  - We write code for **humans**, not computers
  - Messy (**spaghetti**) code runs fine **until requirements change** or bugs appear, then it becomes **hard** to modify, debug, or extend
- **Goal**
  - Design a **Maintainable**, **Flexible**, **Readable** Software Solution
- **Use Case Example**
  - **Before**
    - Payment logic hard coded → adding PayPal requires rewriting everything
  - **After**
    - Payment logic abstracted → just create a **PayPalPayment** class and plug it in

Design Principles: *Don’t Repeat Yourself (DRY)*

- Avoid duplicating logic or knowledge
- Every piece of logic should live in **one place** so changes are **easy** and **consistent**

```java
// In ShoppingCart.java
public double getCartTotal() {  no usages
    double tax = subtotal * 0.05;
    return subtotal + tax + 5.99;
}

// In Checkout.java
public double getFinalTotal() {  no usages
    double tax = subtotal * 0.05; // REPEATED Logic
    return subtotal + tax + 5.99;
}
```

```java
class Utility {  no usages
    // In ShoppingCart and Checkout
    // double total = Utility.calculateTotal(subtotal);
    public static double calculateTotal(double subtotal) {  no usages
        double tax = subtotal * 0.05; // Logic is in ONE place
        return subtotal + tax + 5.99;
    }

    // In Register and Login
    // if(!Utility.validEmail(email)) {...}
    public static boolean validEmail(String email) {  no usages
        return email.endsWith("@gmail.com");
    }
}
```

- **Use Case Example**
  - **Instead of copy-pasting the commission calculation or validation logic across multiple classes, move them into a single reusable utility class**

- Change something **once** → system updates **everywhere** (**Faster updates, Easier maintenance**)

Design Principles: *Keep It Simple, Stupid (KISS)*

- Choose the simplest solution that gets the job done
- Don’t use over-complicated technologies unless they’re truly needed

```
int sum = numbers.stream() Stream<Integer>
        .mapToInt(Integer::intValue) IntStream
        .sum();
```

```
int sum = 0;
for (int num : numbers) {
    sum += num;
}
```

```
return (list != null && !list.isEmpty()) ?
       list.get(0) : (alternative != null ? alternative : "Default");
```

```
if (list != null && !list.isEmpty()) {
    return list.get(0);
}
if (alternative != null) {
    return alternative;
}
return "Default";
```

- **Use Case Example**
  - Do not build a Microservices architecture project with an API Gateway to build a simple CRUD Project for few users

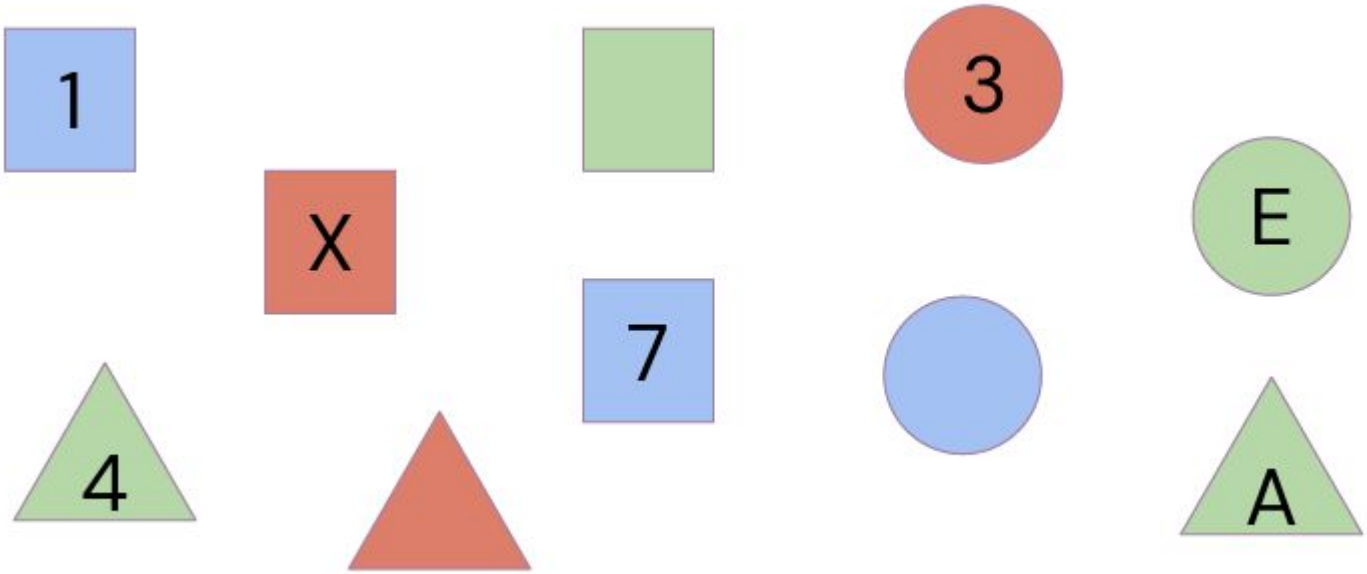
Design Principles: *You Ain't Gonna Need It (YAGNI)*

- Don't build features **just in case**
  - additional unused code increases **code bloat** that increases **testing time** and **system complexity**
- Only implement what you actually need right now

```java
class User {  no usages

    private String username;  1 usage
    private String email;  1 usage

    // Assumed future features (not required now)
    private String phoneNumber;  1 usage
    private boolean phoneVerified;  1 usage
    private String preferredLanguage;  1 usage
    private String facebookLink;  1 usage
    private String twitterLink;  1 usage

    // Constructor with unnecessary parameters
    public User(String username, String email, String phoneNumber,
                boolean phoneVerified, String preferredLanguage,
                String facebookLink, String twitterLink) {

        this.username = username;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.phoneVerified = phoneVerified;
        this.preferredLanguage = preferredLanguage;
        this.facebookLink = facebookLink;
        this.twitterLink = twitterLink;
    }
    //A lot of getters & setters for unused properties…
}
```

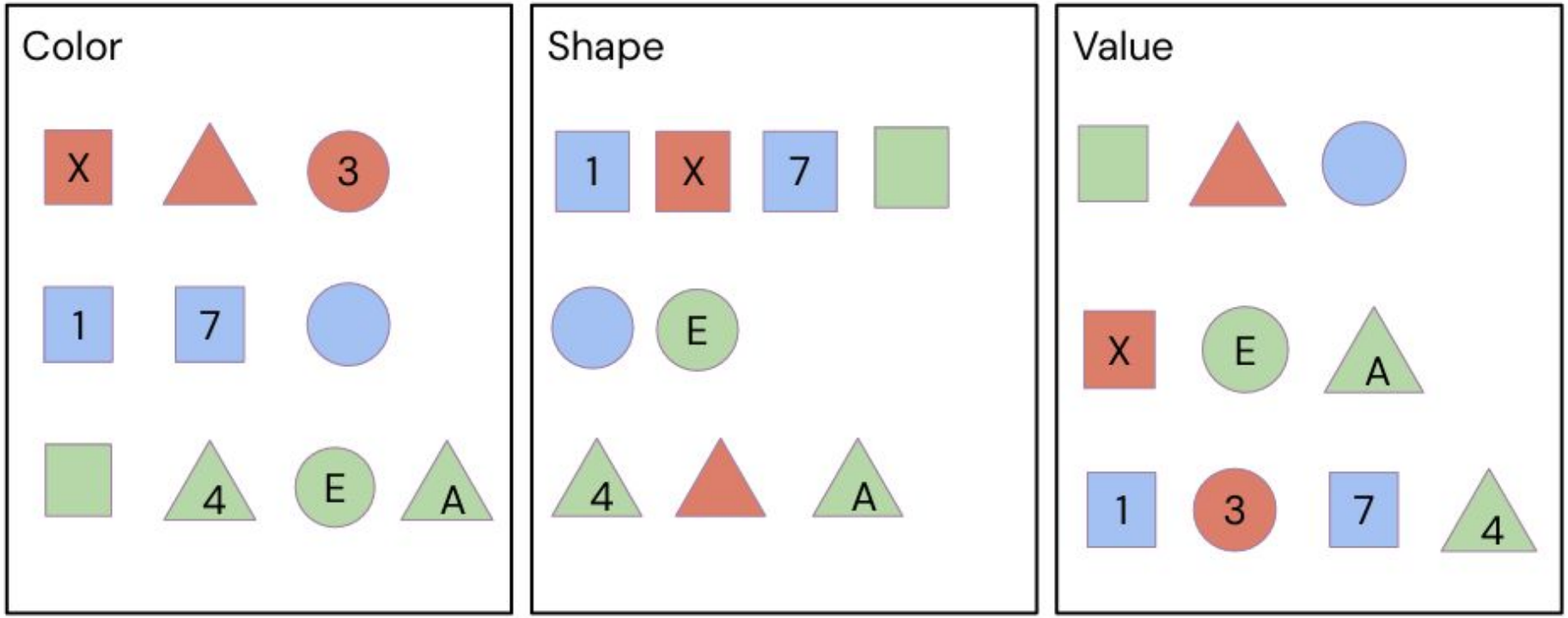
```java
class User {  no usages

    private String username;  2 usages
    private String email;  2 usages

    public User(String username, String email) {
        this.username = username;
        this.email = email;
    }

    // Only required getters
    public String getUsername() {  no usages
        return username;
    }

    public String getEmail() {  no usages
        return email;
    }
}
```

Exercise: Arrange

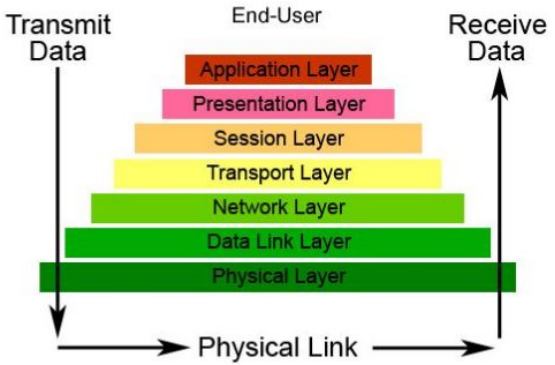


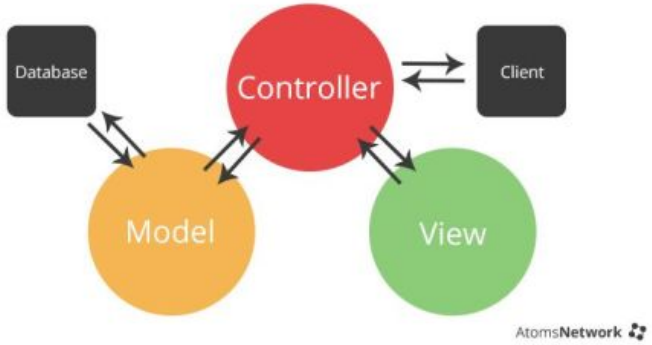
From Eng Hany slides

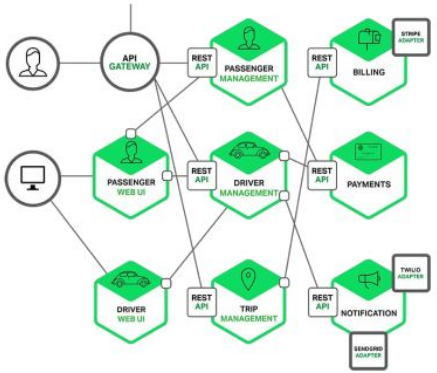
Design Principles: *Separation of Concerns (SoC)*

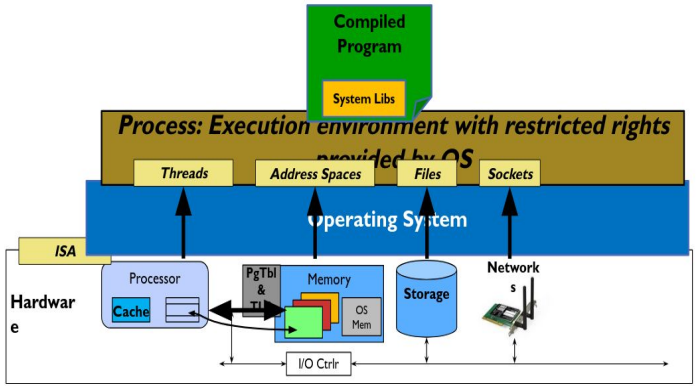


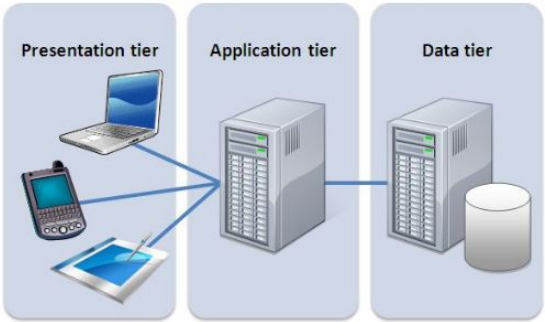
Design Principles: *Separation of Concerns (SoC)*

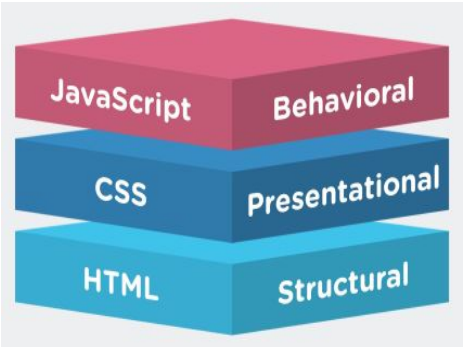












Design Principles: *Separation of Concerns (SoC)*

```
class AuthController {  no usages
    private AuthService service = new AuthService();  1 usa
    public void login(String email, String pass) {  no usag
        service.authenticate(email, pass);
    }
}

class AuthService {  2 usages
    private UserRepository repo = new UserRepository();  1
    public void authenticate(String email, String pass) {
        // business logic
        repo.findUser(email);
    }
}

class UserRepository {  2 usages
    public void findUser(String email) {  1 usage
        // DB logic
    }
}
```

Design Principles: *Encapsulate What Varies*

- Separate what stays the same from what changes frequently

- **Why?**
  - Changing logic is often reused → duplicating it breaks **DRY**
  - Future changes become painful if the logic is scattered everywhere → **Inflexible**
  - Fixing one bug in multiple places → **Rigidity**

```
class PizzaOrder {  no usages

    public double calculatePrice(String pizzaType, int quantity) {  no us
        double basePrice = 0;

        if (pizzaType.equals("MARGHERITA")) basePrice = 80;
        else if (pizzaType.equals("PEPPERONI")) basePrice = 100;
        else if (pizzaType.equals("VEGETARIAN")) basePrice = 90;

        double tax = Constants.PIZZA_TAX;

        return (basePrice * quantity) + ((basePrice * quantity) * tax);
    }
}
```

```
class PizzaPricing {  2 usages

    public double getPrice(String pizzaType) {  1 usage
        return switch (pizzaType) {
            case "MARGHERITA" -> 80.0;
            case "PEPPERONI"  -> 100.0;
            case "VEGETARIAN" -> 90.0;
            default           -> 60.0;
        };
    }
}

class PizzaOrder {  no usages

    private PizzaPricing pricing = new PizzaPricing();  1 usage

    public double calculatePrice(String pizzaType, int quantity) {
        double base = pricing.getPrice(pizzaType) * quantity;
        return base + base * Constants.PIZZA_TAX;
    }
}
```

Design Principles: *Program to Interface, Not Implementation*

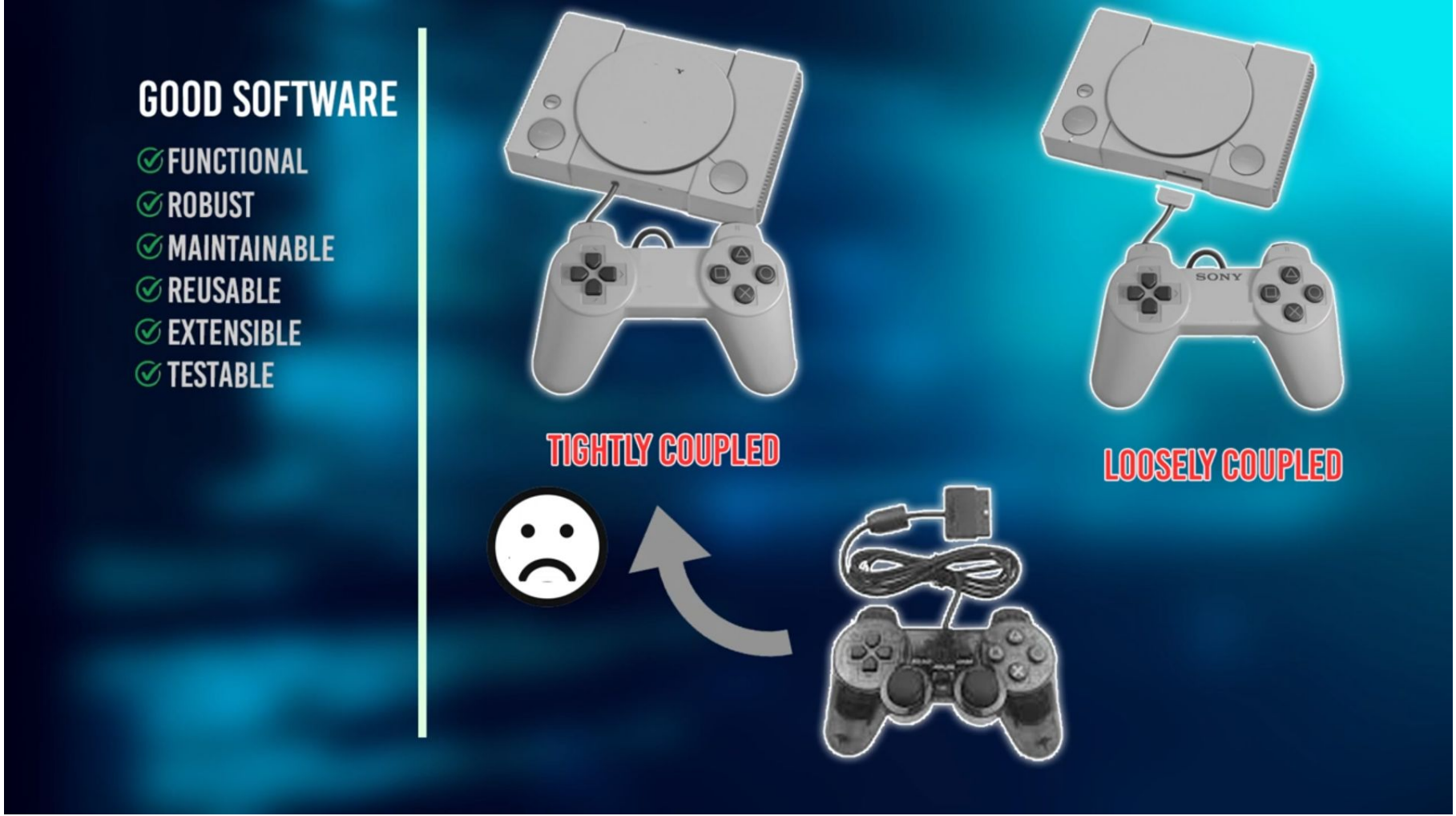
- Write code that **talks to interfaces**, not concrete classes
- High-level modules should depend on **abstractions**, not details
- change **how things work** without changing **who uses them**

- **Why?**
  - Easy to swap implementations (e.g., different payment providers)
  - Better **testability** (you can plug in mocks/fakes)
  - Improves **flexibility** and supports **OCP**

- **Use Case Example**
  - Repository interface with MySqlUserRepository, InMemoryUserRepository
  - NotificationSender interface with EmailSender, SmsSender, PushSender

Hand On Time

Tightly Coupled vs Loosely Coupled


GOOD SOFTWARE
FUNCTIONAL
ROBUST
MAINTAINABLE
REUSABLE
EXTENSIBLE
TESTABLE
TIGHTLY COUPLED
LOOSELY COUPLED
SONY

src

Design Principles: *Tight Coupling*

- Components are heavily dependent on each other's internal implementation details
- Changes to one component often require changes to multiple other components

- **Characteristics**
  - Direct dependencies on concrete classes rather than interfaces
  - Components know too much about each other's internal workings
  - Hard-coded references between modules
  - One component directly instantiates another

- **Problems**
  - Difficult to modify (**Inflexibility**)
  - Hard to test and maintain
  - Reduced reusability
  - Parallel development issues

Design Principles: *Loose Coupling*

- Loose coupling minimizes dependencies between components
- Each component knows as little as possible about others

- **Characteristics**
  - Dependencies on abstractions (interfaces) rather than concrete implementations
  - Components can be developed, tested, and deployed independently
  - Minimal shared state between components
  - Use of dependency injection or service locators
- **How to Achieve? some**
  - Dependency Injection
  - Interface Segregation
  - Event-Driven Architecture
  - Configuration Externalization

Design Principles: *Favor Composition Over Inheritance*

- Choose composition instead of building **deep inheritance trees**
- Composition gives more **flexibility**, **reusability**, and **runtime configurability**

- **Why?**
  - Remember Inheritance Problem?
    - Inheritance forces **rigid IS-A** relationships
    - Hard to change or extend later
    - Hierarchies **grow fast** and become unmaintainable
    - You cannot **combine** multiple behaviors easily
  - Composition lets you **mix behaviors dynamically** instead of baking them into a class

```
class Pizza {}  2 usages  3 inheritors
class MargheritaPizza extends Pizza {}  no usages
class MexicanPizza extends Pizza {}  1 usage  1 inheritor
class SpicyMexicanMargheritaPizza extends MexicanPizza {}
```

```
interface Ingredient {  5 usages  2 implementations
    String getName();  no usages  2 implementations
    double getPrice();  1 usage  2 implementations
}

class Cheese implements Ingredient {  no usages
    public String getName() { return "Cheese"; }  no usages
    public double getPrice() { return 10; }  1 usage
}

class Sauce implements Ingredient {  no usages
    public String getName() { return "Sauce"; }  no usages
    public double getPrice() { return 5; }  1 usage
}

class Pizza {  no usages
    private List<Ingredient> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient i) {  no usages
        ingredients.add(i);
    }

    public double getTotalPrice() {  no usages
        return ingredients.stream()  Stream<Ingredient>
                .mapToDouble(Ingredient::getPrice)  DoubleStream
                .sum();
    }
}
```

Caller

```
Pizza pizza = new Pizza();
pizza.addIngredient(new Cheese());
pizza.addIngredient(new Sauce());
System.out.println(pizza.getTotalPrice());
```

Thank You